

Python Tutorial

Veja nesse artigo um tutorial de introdução a linguagem de programação Python com as suas sintaxes de forma prática, resumida e exemplificada.

[Artigos](#) > [Canal Mais](#) > [Python Tutorial](#)

Python é uma linguagem de programação multiplataforma que permite desenvolver aplicações para games, desktops, web e dispositivos móveis. Além disso, ela pode se comunicar com outras aplicações que foram desenvolvidas em outras linguagens como C, C++, [Java](#) e [C#](#). Nesse artigo serão vistos exemplos diversos de sintaxes básicas utilizando o Python.

Relacionado: [Curso de Python](#)

Conceito de Variável em Python

O **conceito de variável em Python** é representado sempre por um objeto (tudo é objeto), e toda variável é uma referência. Na maioria das linguagens de programação, quando iniciamos uma variável e atribuímos um valor a ela, essas carregam valores que são alocados em memória, e quando alteramos os seus valores, estamos alterando o valor na memória também. Porém, no Python as variáveis armazenam endereços de memória e não os valores.

Outro ponto a considerar é que em Python uma variável não tem um tipo fixo, mas sim apenas o tipo do conteúdo, como mostra o exemplo da **Listagem 1**.

```
1 x = [1,2,3]
2 y = x
3 x.append(4)
4 print(y)
```

Listagem 1. Saída gerada: [1, 2, 3, 4]

Na linha 1 é criada uma variável `x` que recebeu um vetor de inteiros. Em seguida, foi referenciada uma variável `y` onde é atribuída a referência de `x`. Na linha 3 observamos que `x` também é um objeto quando invocamos o método `append`, que adiciona um elemento ao vetor `x`. Na linha 4 utilizamos o método `print()` para exibir a variável `y`. Em Python, uma variável não tem um tipo fixo, apenas o tipo do conteúdo atual, por isso o vetor se atualizou e assim o valor de `y`.

Em relação ao nome que podemos atribuir as variáveis, pode-se utilizar letras, o caractere de sublinhado `_` e números. Os comentários em Python são representados pelo símbolo `#` e colocado em cada linha.

Tipos de Dados em Python

Uma característica que facilita muito a vida do desenvolvedor são as conversões de tipos em Python. Veja na **Listagem 2** alguns exemplos.

```
1  a = float(22/5)
2  b = int(4.5)
3  c = int(3.9)
4  d = int(0xff563)
5  e = float(int(3.9))
6  f = int(float(3.9))
7  g = int(float(3))
8  h = round(3.9)
9  i = round(3)
10 j = int(round(3.9))
11 print(a,b,c,d,e,f,g,h,i,j)
```

Listagem 2. Alguns exemplos de conversões

Vamos entender melhor cada tipo:

Inteiros

Para a declaração de números inteiros, precisamos que estes estejam entre `-2147483648` a `2147483647`. Cada um ocupa quatro bytes na memória e pode armazenar tantos valores decimais (de base 10), quanto valores octais (de base 8) e hexadecimais (base 16). Para declarar um inteiro octal, o número `0` (zero) tem que ser prefixado ao número, como em `0123`; e para definir um número hexadecimal, o prefixo `0x` ou `0X` deve ser utilizado, como `0xFFFF` ou `0X006699`. Observe o exemplo na **Listagem 3**.

```
1 | a = 42 #decimal
2 | b = 010 #octal
3 | c = 0xA #hexadecimal
```

Listagem 3. Exemplo utilizando os tipos: decimal, octal e hexadecimal

Na versão 3.7 do Python, os números octais devem obrigatoriamente ter o prefixo `0o`, pois essa versão não suporta o modelo anterior. Ao invés de usar `'010'`, deve-se usar `'0o10'` na versão 3.7

Long

Representa números inteiros longos e pode armazenar números tão grandes quanto a memória puder armazenar. Assim como o tipo `int`, o `long` também pode armazenar números tanto decimais, quanto octais e hexadecimais. Para declarar um valor `long` mesmo se o número a ser atribuído estiver na faixa de valores do tipo `int` é necessário sufixar a letra L (minúscula ou maiúscula) como em `5245115743621`, `0xDDEFBDAEFBDAECBFBAEL` e `0122L`.

Exemplo: `a = 0xDDEFBDAEFBDAECBFBAEL` representa: `hexadecimal Long`

Float

Representa números reais e que possuem sinal de expoente (e ou E). Esses números são comumente chamados de `floating-point numbers` ou números de ponto flutuante. Por exemplo: `0.0042`, `.005`, `1.14159265` e `6.02e23` (o mesmo que 6.02×10^{23}).

Bool

O tipo `bool` foi adicionado na versão 2.2 do Python como uma especialização do tipo `int`. Os valores do tipo `bool` podem representar dois valores completamente distintos: `True` (igual ao `int` 1) e `False` (igual ao `int` 0) para, respectivamente, verdadeiro e falso. Exemplo: `a = True` e `b = False`.

None Type

NoneType é o tipo de `None`, uma constante embutida do Python que, assim como `True` e `False`, e é frequentemente utilizada para representar a ausência de um valor, similar ao `null` na linguagem C e derivadas. Exemplo: `a = None` (o mesmo que `null` em Java).

String

Para atribuímos a uma variável uma referência do tipo string, basta que coloquemos entre aspas simples, duplas ou triplas, como mostra a **Listagem 4**.

```
1 | a = 'Isso é uma String com aspas Simples'
2 | b = "Isso é uma String com aspas Duplas"
3 | c = """Isso é Uma String com aspas Triplas"""
4 | print(a + "\n" + b + "\n" + c)
```

Listagem 4. Exemplos com strings

Este código gera uma saída com cada frase em uma linha nova, mesmo estando concatenada. Isso acontece porque foi utilizado o recurso de quebra de linha com o `\n`.

Em Python, tudo é objeto, assim as Strings são objetos que tem embutidos vários métodos. Vamos analisar o seguinte código da **Listagem 5**.

```
1 1 #coding: utf-8
2 2 meu_nome = "Sintaxes Python"
3 3 meu_nick = 'Devmedia'
4 4 print ("Nome: %s, Nick: %s" % (meu_nome.upper(), meu_nick))
5 5 print ("Meu nome começa com a letra ", meu_nome[0])
6 6 print ("Meu nome começa com a letra ", meu_nome[0].lower())
7 7 print ("Meu primeiro nome é ", meu_nome[0:7])
```

Listagem 5. Trabalhando com strings

Na linha 1 é utilizado um comentário `#coding: utf-8`, que permite a acentuação.

Nas linhas 2 e 3 são criadas variáveis as quais atribuímos uma string. Na linha 4 utilizamos um recurso muito conhecido em C, no qual exibimos uma string e indicamos quais os tipos de variáveis que serão utilizadas naquela referência, no caso de string utiliza-se o `%s`. Note que a exibição é respectiva a sequência da declaração na mesma linha. Use o método `upper()` para deixar todas as letras em caixa alta.

Na linha 5 exibimos novamente a concatenação de uma string com a variável que é um objeto, contudo, sabemos que string são vetores de caracteres e utilizando a variável seguida de um índice o compilador mostra apenas a primeira posição do vetor. Na linha 6 esse mesmo vetor ainda invoca o método `lower()` que mostra o caractere em caixa baixa, e na linha 7 esse vetor é exibido da sua posição `0` até a posição `7`.

Sobre a linha 4 na **Tabela 1** temos os tipos de variáveis para os formatos dos símbolos e suas respectivas conversões.

Formato do Símbolo	Conversão
%c	Caractere
%s	String convertida pelo método str() tem prioridade
%i	Sinal decimal inteiro
%d	Sinal decimal inteiro
%u	Decimal inteiro sem negativos (unsigned)
%o	Octal inteiro
%x	Hexadecimal Inteiro (letras em caixa baixa)
%X	Hexadecimal Inteiro (Letras em Caixa ALTA)
%e	Notação exponencial (Para letras minúsculas 'e')
%E	Notação exponencial (Para letras MAIÚSCULAS 'E')
%f	Ponto flutuante (Números reais)
%g	O mais curto de 'f%' e 'e'
%G	O mais curto de 'f%' e 'E'

Tabela 1. Tabela que representa alguns formatos de símbolos e sua conversão de acordo com a referência para os tipos de variáveis

Outros métodos interessantes são o `capitalize()`, que faz com que apenas o primeiro caractere seja exibido em caixa alta. O método `strip()` retorna uma cópia da string com os espaços em branco antes e depois removidos. O método `startswith`(prefixo) retorna `True` se uma string começa com o prefixo passado por argumento, e `False` caso contrário.

Operadores

A **Tabela 2** resume como são utilizados os operadores na linguagem.

Aritméticos	Comparação	Lógicos
+	==	and
-	!=	or
*	>	not
/ ou // (parte inteira)	<	
%	>=	
+= -= *= /=	<=	
**	in in not	
is		

Tabela 2. Tabela com valores aritméticos, comparação e lógicos (e/ou/não)

Entrada de Dados no Python

Em **Python**, a leitura de dados do teclado é feita através das funções `raw_input` e `input`, como mostra o exemplo da **Listagem 6**. Contudo, em versões superiores a 3.0, a função `raw_input` não funciona.

```
1 1 import sys
2 2
3 3 nome = input("Digite seu nome: ")
4 4 idade = input("Digite sua idade: ")
5 5 print("Digite seu sexo: ")
6 6 sexo = sys.stdin.readline()
7 7
8 8 print("Nome:"+nome + "\n" + "Sexo: %s Idade: %s" %(sexo,idade))
```

Listagem 6. Função input

Na linha 8 são utilizadas duas maneiras para exibir as strings, e também são utilizadas duas maneiras de fazer a leitura do teclado: uma diretamente na variável com exibição de uma mensagem, e outra em que utilizamos um método `readline()`, que está contido na importação feita na linha 1.

Comandos de Decisão

Para quem já tem contato com alguma linguagem de programação, é possível notar que é muito simples os comandos **condicionais de Python**:

```
1  if CONDIÇÃO :  
2      BLOCO DE CÓDIGO  
3  elif CONDIÇÃO :  
4      BLOCO DE CÓDIGO  
5  else :  
6      BLOCO DE CÓDIGO
```

IF, ELIF, ELSE no Python

Uma observação que deve ser feita é que em Python não usamos as chaves `{ }` para iniciar e finalizar um bloco de instruções. Para que você delimite onde cada bloco começa e acaba, basta indentar o código, como mostra a **Listagem 7**.

```
1  # coding:utf-8  
2  dedos = int(input("Você tem quantos anos? "))  
3  
4  if dedos == 18:  
5      print("Você tem 18 anos")  
6  elif dedos > 18:  
7      print("Você tem mais de 18 anos")  
8  else:  
9      print("Você é menor de idade")
```

Listagem 7. Exemplo básico de como pode ser usado os comandos de decisões: If, Elif e

Else

Switch Case

Outra forma de utilizar o `if`, `elif` e `else`, é implementando-o como uma estrutura `switch`, onde cada `elif` vai ter um case dentro do `switch`, como mostra o exemplo da **Listagem 8**.

```
1  # coding: utf-8
2  var1 = int(input("Digite um Número para var1"))
3  var2 = int(input("Digite um Número para var2"))
4  if var1 == 1:
5      print("Número var1 igual a 1")
6  elif var1 == 2 or var2 == 3:
7      print("var1 diferente de 1 ou var2 diferente de 2")
8  elif var1 >= 1000 or var2 <= -1000:
9      print("var1 maior que 1000 ou var2 menor que -1000")
10 else:
11     print("nenhuma das alternativas anteriores")
```

Listagem 8. Exemplo de uma estrutura idealizada como switch, onde lê os dados e os compara de acordo com as condições, exibindo mensagem para o usuário

Função RANGE()

Existe também a função `range()`, que retorna um `array`, ou seja, os elementos são representados em uma sequência, como mostra a **Listagem 9**.

```
1 | 1 print (range(10)) # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
2 | 2 print (range(5, 10)) # [5, 6, 7, 8, 9]
3 | 3 print (range(10, 0)) # [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
4 | 4 i=int(input("Digite um Número"))
5 | 5 if i in range(0,10):
6 | 6     print("Está Contido")
7 | 7 else:
8 | 8     print("Não está Contido")
```

Listagem 9. Exemplo utilizando range

Nas linhas 1 a 3 é impresso na tela os valores (que estão em comentários), e a partir da linha 4 é verificado se o valor digitado pelo usuário está contido na sequência de 0 a 9.

Laço de Repetição FOR

No laço de repetição for é **implementado em Python** e pode ser utilizado de diversas maneiras, como exibido na **Listagem 10**.

```
1 1 #coding: utf-8
2 2 for fruta in [ "banana", "maca ", "uva" ]:
3 3     print ("Fruta : " + fruta)
4 4 print("-----")
5 5 for i in range (0,10): #(Inclui o Zero, NÃO INCLUI O 10)
6 6     print ("i = " + str ( i ))
7 7 print("-----")
8 8 for i in range(0,10):
9 9     print ('Não realizado1')
10 10         if i == 0:
11 11             break
12 12 print ('Realizado')
```

Listagem 10. Laço For

Nesse exemplo, o for da linha 2 armazena três valores para a variável `fruta`.

No for da linha 5 armazena os valores da sequência range para a variável `i` e dentro do for da linha 8 demonstra que é possível colocar estruturas de condições.

Laços de repetição WHILE no Python

O laço de repetição `while` no Python tem basicamente a mesma estrutura do `for`, porém, como no `for` ou `if`, não se utilizam os parênteses para definir a condição e seu bloco de instrução também é definido com a indentação, como mostra a **Listagem 11**.

```
1 1 i = 0
2 2 while i < 10:
3 3     print ("i = ",i)
4 4     i += 1
5 5
6 6 while True:
7 7     pass
```

Listagem 11. Exemplo de utilização do `while`

A instrução `pass` faz nada, ou seja, se quisermos definir um ciclo ou função que não tenha nenhuma instrução a ser executada, usamos o `pass`.

Exceções e Listas

As exceções podem ser tratadas utilizando o recurso do `try...catch`, que tem como sintaxe o código da **Listagem 12**.

```
1 try:
2     print (1 + 'st try')
3 except:
4     print ("Isto é uma exceção")
```

Listagem 12. Exemplo básico de como pode ser utilizado try e except

Uma lista é um conjunto de valores indexados por um número inteiro chamado de índice, que se inicia em zero.

Os elementos contidos em uma lista podem ser de qualquer tipo, até mesmo outras listas, e não precisam ser todos do mesmo tipo. Uma lista é delimitada por colchetes e seus elementos separados por vírgulas, como vemos no exemplo da **Listagem 13**.

```
1 lista = [1, 2, 3]
2 print ("Tamanho de lista:", len(lista))
3
4 lista[0]=9
5 print("-----")
6 for i in range(len(lista)):
7     print (lista[i])
```

Listagem 13. Trabalhando com listas

Utilizamos um método que recebe uma lista por parâmetro e devolve o tamanho da lista. Note que a função `range()` também existe no laço `for`. Na linha 4 é exibido o item `0` da lista.

Veja na **Listagem 14** execução e o resultado de como acessar o conteúdo das listas em Python. Repare que ao usar índices negativos, as posições são acessadas a partir do final da lista.

```
1 lista = [1, 2, 3]
2 print (lista[0])
3 print (lista[1])
4 print (lista[-1])
5 print (lista[-2])
6 print (lista[0:2])
7 print (lista[:2])
8 print (lista[1:])
```

Listagem 14. Executando listas

- Linha 2 imprime 1;
- Linha 4 imprime valor 3;
- Linha 5 imprime 2;
- Linha 6 imprime [1,2];
- Linha 7 imprime [1,2];
- Linha 8 imprime [2,3].

A função `capitalize()` deixa a string em caixa alta. Na **Listagem 15** essa função está sendo aplicada em uma lista e dentro de um laço de repetição.

A função `capitalize()` deixa a string em caixa alta. Na **Listagem 15** essa função está sendo aplicada em uma lista e dentro de um laço de repetição.

```
1  frutas=['laranja','banana','abacaxi']
2
3  frutas.sort()
4  print (frutas)
5  frutas[0]='framboesa';
6
7  for item in frutas:
8      print (item.capitalize())
```

Listagem 15. Código responsável por criar uma cópia da string e retorná-la com o primeiro caractere em caixa alta

A função `remove()` tira um item de uma lista em Python, como mostra o exemplo na **Listagem 16**.

```
1 1 convidados = [] #define uma lista vazia
2 2 convidados.append ('Exemplo 1')
3 3 convidados.append (' Exemplo 2')
4 4 convidados.append (' Exemplo 3')
5 5 convidados.append (' Exemplo 4')
6 6 print ("Tenho ", len(convidados)," Convidados")
7 7 convidados.sort()
8 8 print ("Sao eles:")
9 9 print (convidados)
10 10 print ("O primeiro convidado eh o ",convidados[0])
11 11 convidados.remove("Exemplo 4"); #aqui tiramos o Exemplo 4 da lista
12 12 print ("Agora tenho somente ", len(convidados)," Convidados")
13 13 print ("Sao eles:")
14 14 for convidado in convidados:
15 15     print (convidado)
```

Listagem 16. Trabalhando com lista

O código é responsável por criar uma lista adicionando elementos, exibir os convidados e remover um deles, exibindo logo após uma lista atualizada dentro do laço for.

Tuplas e Dicionários

Até agora foram vistos dois tipos compostos: strings, que são compostos de caracteres; e listas, que são compostas de elementos de qualquer tipo. Uma das diferenças que notamos é que os elementos de uma lista podem ser modificados, mas os caracteres em uma string não. Em outras palavras, strings são imutáveis e listas são mutáveis.

Há outro tipo em `Python` chamado `tupla` (tuple), que é similar a uma lista, exceto por ser imutável. Sintaticamente, uma tupla é uma lista de valores separados por vírgulas e com aspas simples, como vemos na **Listagem 17**.

```
1  frutas=('laranja','banana','abacaxi')
2
3  frutas.sort()
4  print (frutas)
5  frutas[0]='framboesa';
6  for item in frutas:
7      print (item.capitalize())
```

Listagem 17. Código que armazena uma lista de valores, ordena, exibe e os transforma em caixa alta dentro do laço for

Tuplas e Dicionários

Até agora foram vistos dois tipos compostos: strings, que são compostos de caracteres; e listas, que são compostas de elementos de qualquer tipo. Uma das diferenças que notamos é que os elementos de uma lista podem ser modificados, mas os caracteres em uma string não. Em outras palavras, strings são imutáveis e listas são mutáveis.

Há outro tipo em `Python` chamado `tupla` (tuple), que é similar a uma lista, exceto por ser imutável. Sintaticamente, uma tupla é uma lista de valores separados por vírgulas e com aspas simples, como vemos na **Listagem 17**.

```
1  frutas=('laranja','banana','abacaxi')
2
3  frutas.sort()
4  print (frutas)
5  frutas[0]='framboesa';
6  for item in frutas:
7      print (item.capitalize())
```

Listagem 17. Código que armazena uma lista de valores, ordena, exibe e os transforma em caixa alta dentro do laço for

Os **dicionários em Python** são **estruturas de dados** que permitem “traduzir” uma chave para um valor, ou seja, são estruturas de dados que implementam mapeamentos.

Um mapeamento é uma coleção de associações entre pares de valores, onde o primeiro elemento do par é chamado de chave e o outro de conteúdo. De certa forma, o mapeamento é uma generalização utilizada para acessar os dados das listas pelas suas chaves que funcionariam como índices, contudo estas chaves necessitam ser imutáveis. Vejamos um exemplo na **Listagem 18**.

```
1 1 veiculo = {}
2 2 veiculo['marca'] = 'Puma'
3 3 veiculo['modelo'] = 'GTB'
4 4 veiculo['ano'] = 1978
5 5 print (veiculo['marca'])
6 6 dicionario = {"chave": "17,532", "chave2": "17,365" }
7 7 print (dicionario ["chave"]) #imprime valor (17,532)
```

Listagem 18. Exemplo de tupla

Esse código representa as declarações de listas com atributos de um carro, como marca. A variável dicionário recebe as chaves mapeadas e depois imprime a que está na primeira posição.

Matriz com Listas no Python

Em Python o conceito de `arrays` multidimensionais não existe. O que existe é uma alternativa, que são os `arrays` encadeados, ou seja, cada elemento do `array` corresponde a outro `array`, ou como uma `list` de `list` em Java.

Veja um exemplo, na **Listagem 19**, de como pode ser criada uma matriz de 10 linhas e cinco colunas e como ela pode ser “varrida”.

```
1 1 #Criando uma matriz de 10x5 inicializada com 0
2 2 lin=10
3 3 col=5
4 4 matriz=[]
5 5 for i in range(0,lin):
6 6     linha=[]
7 7     for j in range(0,col):
8 8         linha.append(0)
9 9     matriz.append(linha)
10 10 #varrendo a matriz
11 11 for i in range(0,len(matriz)):
12 12     for j in range(0,len(matriz[0])):
13 13         print (matriz[i][j])
```

Listagem 19. Trabalhando com matriz

O primeiro for (onde as linhas são adicionadas, logo após terminar de percorrer as colunas em uma variável que é criada localmente) é definido especificando o começo incluso e a linha não inclusa, pois um vetor inicializa na posição zero. No for mais interno do primeiro exemplo nota-se o mesmo uso, onde este adiciona zeros nas colunas, retorna ao laço mais externo e adiciona mais uma linha.

Funções

Assim como em outras linguagens, uma função pode receber alguns parâmetros e devolver um resultado. A sintaxe inicia-se com a palavra `def`, seguida do nome da função e dos parênteses que podem conter seus parâmetros. As funções também são consideradas objetos e têm alguns atributos, dentre os quais os mais úteis são `__doc__`, o qual contém a documentação da função, e `__name__`, que contém o nome (apelido) da função.

Mas antes de entrar diretamente no código, é importante abordar sobre a recursividade da linguagem e sobre a documentação da função. Recursividade de uma função basicamente é a definição de uma sub-rotina, seja ela função ou método, que tem a capacidade de invocar a si mesmo como, por exemplo, o fatorial.

Outro recurso é a documentação, que nada mais é do que um comentário dentro da função que é exibido quando utiliza-se o método `__doc__`, como vemos no caso da

Listagem 20.


```

1 1 def fatorial (numero): #função recursiva
2 2     """
3 3     Funcao recursiva
4 4     As três aspas duplas é a documentação
5 5     """
6 6     if numero <= 1:
7 7         return 1
8 8     else:
9 9         return (numero * fatorial (numero - 1 ))
10 10 for n in range ( 1, 11 ):
11 11     print ("Fatorial de",n," eh ", fatorial(n))
12 12 print ("\nDoc. da função:\n"+fatorial.__doc__)

```

Listagem 20. Código que representa uma função recursiva

A função retorna o valor do fatorial do número que está dentro do `for`. A linha 12 imprime a documentação que está dentro da função.

Também é possível **utilizar função em Python** com parâmetros opcionais como, por exemplo, uma função que calcula a potência, onde poderíamos ou não informar o expoente. Caso não fosse informado assumira o padrão de 2, como no código da **Listagem 21**.

```
1 1 def potencia(base,exp=2): # função com parâmetro opcional
2 2 if exp==0:
3 3     return 1
4 4 pot=base;i=1;
5 5 while i < exp:
6 6     pot=pot*base
7 7     i=i+1
8 8 return pot
9 9 print("Começa aqui!")
10 10 print (potencia(2)) # imprime 4
11 11 print (potencia(2,8)) #imprime 256
```

Listagem 21. Função que eleva um número (base) a uma potência (exp)

Caso não seja informado o expoente, será considerado o expoente 2. A linha 10 imprime valor 4 e a linha 11 imprime valor 256.

Arquivos no Python

Manipular arquivos em `Python` é extremamente simples, como vemos na **Listagem 22**.

```
1 | 1 arq=open('D:/meuarquivo.txt', 'w')
2 | 2 arq.write('gravando em um arquivo é simples')
3 | 3 arq.close ()
```

Listagem 22. Manipulando arquivos

Um arquivo aberto é um objeto instanciado pela função `open()`. Repare que na linha 1 é passado por parâmetro o local do arquivo (foi utilizado o disco D: para simplificar o exemplo). Dependendo da configuração pode dar um erro referente aos privilégios de gravação, mas se isso ocorrer, utilize uma pasta comum do usuário logado. Como segundo argumento é passado o 'w', que indica a criação de um novo arquivo para a escrita.

Na **Tabela 3** vemos a lista de argumentos que podemos usar.

Argumento	Ação
W	Sempre cria um novo arquivo para escrita.
w+	O mesmo que “w”, mas abre para escrita e leitura.
R	Abre para leitura.
A	Abre o arquivo para escrita, mantendo o conteúdo e posiciona o cursor no final do arquivo, utilizado para adicionar conteúdos.
a+	O mesmo que ‘a’, mas abre para leitura e escrita.
B	Em combinação com os modos acima, manipula os arquivos binários (wb, rb, ab+,...).

Tabela 3. Argumentos que podem ser utilizados para manipulação de arquivos

A classe utilizada para executar as operações é a `file`, e assim que é executado o comando `open` é retornado um objeto do tipo `file`. Na **Listagem 22** foram vistos três exemplos de métodos possíveis com arquivo: o `open`, `write` (escreve no arquivo) e `close` (que fecha o arquivo).

Para leitura do arquivo existe o método `read`, e se utilizarmos somente, ele nos retorna todo o conteúdo do arquivo, como podemos ver na implementação do código da **Listagem 23**.

```
1 | 1 arq = open('d:/meuarquivo.txt') # r é default
2 | 2 leitura = arq.read(3) aa
3 | 3 print(leitura)
4 | 4 restante=arq.read()
5 | 5 print(restante)
```

Listagem 23. Lendo arquivos

É aberto o arquivo para leitura e é passado por parâmetro a quantidade de caracteres que se deseja ler. Se for utilizado novamente sem parâmetro, ele retorna o restante do texto não lido.

A **linguagem de programação Python** é realmente muito simples, fácil e são utilizadas poucas linhas para se trabalhar, facilitando no desenvolvimento e em seu desempenho.